Undo / Redo for Any Form

Problem: Add undo / redo to a form (text, email, textarea, select, radio, checkbox), with buttons and keyboard shortcuts.

Often asked at: Atlassian, Adobe · **Why it matters:** Command pattern, history stacks

Tech stack: React (Vite, JavaScript, Tailwind CSS) · **Time limit:** 60 minutes

---

PART A: Thought process

The same loop works for every machine coding task:

1. **Clarify requirements**: what must work, what is out of scope. Ask before you build.
2. **List the interactions**: every user action and what the UI must do. This becomes your checklist.
3. **Sketch the architecture**: a few components and one place for the logic.
4. **Design the state**: the smallest state that can describe the UI. Derive everything else.
5. **Define data shapes / contracts**: what a "command" looks like, what the hook returns.
6. **Trace one interaction end to end** to prove the design.
7. **Find the hard parts** and decide how to solve them before coding.
8. **Build in layers**: a working version first, then add one feature at a time, keeping it runnable.
9. **Cover states and edge cases**, then **wrap up** with trade-offs and next steps.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state, contracts	10 min
Core build (form + history reducer + undo/redo buttons)	20 min
Hard parts (typing merge, reset, shortcuts, focus)	15 min
Edge cases, polish, walkthrough with interviewer	10 min

Tip: think out loud. Say the decision and the reason: "I'll store commands, not full snapshots, because a command only holds the diff and undo is just applying `from`." The interviewer grades your reasoning as much as your code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A form with several field types: text, email, textarea, select, radio group, checkbox.
- **Undo** reverts the last change; **Redo** re-applies the last undone change.
- Buttons for Undo / Redo / Reset, disabled when there is nothing to do.
- Keyboard shortcuts: `Ctrl/Cmd+Z` undo, `Ctrl/Cmd+Shift+Z` and `Ctrl+Y` redo.
- Typing a word is **one** undo step, not one per keystroke.
- A new edit after an undo clears the redo history (standard editor behaviour).
- Reset is itself undoable.
- Submit shows the current values.

Non-functional requirements

- **Performance**: no full-form snapshot per keystroke; history capped (100 steps).
- **Reusability**: the undo logic must work for *any* form, not just this one. It lives in a pure reducer + one hook, keyed by the input's `name`.
- **Extensibility**: new field types or new commands (e.g. "fill from template") plug in without touching the UI.
- **Usability / a11y**: focus moves to the field that changed; buttons are real `<button>`s with `aria-keyshortcuts`; disabled states; `aria-live` status for dirty / clean.
- **Error handling**: undo / redo on empty stacks are no-ops; no-op edits don't create history.

Clarifying questions to ask

1. Should one undo revert a whole word / burst of typing, or every single character?
2. Is the history per form or global for the page (e.g. multiple forms)?
3. Should undo history survive a page refresh or a successful submit?
4. Do we need validation? Should undo also restore error messages?
5. Is there a history limit? Do we need a visible history list?
6. Should the browser's native per-input `Ctrl+Z` be replaced by ours?

Assumptions we build with

- Text edits are grouped: consecutive edits to the same field with < 600 ms pause are one step.
- History is per form, in memory only, capped at 100 steps. Submit does not clear it.
- Our shortcut replaces the native per-input undo inside the form.
- No validation (it is a separate concern; errors would be derived from values).
- A small history panel is shown to make the stacks visible (great for the demo).

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Type "Ash" quickly into Name	One history entry `name: "" → "Ash"`
Pause > 600 ms, type "u"	A second entry `name: "Ash" → "Ashu"`
Click Undo / press Ctrl+Z	Name goes back one step, focus moves to Name, entry moves to redo list
Click Redo / Ctrl+Shift+Z / Ctrl+Y	Re-applies the last undone entry
Undo, then type something new	Redo list is cleared; new entry is added

Action / input	Expected behavior / output
Type "a" then Backspace in one burst	No entry at all (net change is zero)
Toggle checkbox twice	Two entries (discrete fields never merge)
Change select or radio	One entry per change
Click Reset	All changed fields return to initial; **one** entry `Reset N fields`
Undo after Reset	Every field comes back at once
Reset on a clean form	Button disabled; no entry
Undo with empty history	Button disabled; shortcut does nothing
150 edits	Only the latest 100 are kept
Submit	Shows current values as JSON; history stays

---

3. Design the architecture

Module	Responsibility
`historyReducer.js`	Pure logic: commands, `past` / `future` stacks, merge, reset, undo, redo. No React.
`useUndoRedoForm(initialValues)`	Binds the reducer to React: generic `handleChange`, shortcuts, focus, derived flags.
`ProfileForm`	The concrete form: fields + toolbar. Knows nothing about stacks.
`HistoryPanel`	Shows applied and undone commands. Pure presentational.
`App`	Page layout.

```
App
└── ProfileForm ── useUndoRedoForm(INITIAL_VALUES)
    │                 └── historyReducer (pure, testable in Node)
    ├── toolbar: Undo / Redo / Reset / dirty status
    ├── fields (name, email, bio, role, plan, newsletter)
    └── HistoryPanel (past, future)
```

Why this split: the **Command pattern** logic is the part interviewers care about, so it sits in a pure reducer you can unit test without a browser. The hook is the only React-aware layer. Any form becomes undoable by calling the hook and wiring `onChange={handleChange}`.

---

4. Design the state / data model

State or field	Purpose
`values`	Current form values (the "present")
`past`	Stack of executed commands; top = next undo
`future`	Stack of undone commands; top = next redo
`canMerge`	`true` only right after a new edit; `false` after undo / redo, so typing never merges into an old command

```
const createHistory = (values) => ({ values, past: [], future: [], canMerge: false });

const [state, dispatch] = useReducer(historyReducer, initialValues, createHistory);
```

Senior note: derive, don't store.

- `canUndo = past.length > 0`, `canRedo = future.length > 0`.
- `isDirty = some field !== initialValues[field]`. Not `past.length > 0`: you can edit and then edit back.
- The history labels in the panel are computed from commands at render time.
- **Ownership**: the hook owns all history state; `ProfileForm` owns only `submitted` (UI-only). The reducer owns the rules.

Why one reducer and not `useState` × 4: undo must change `values`, `past` and `future` **atomically**. One reducer = one transition, no torn states.

---

5. Design the data structures and contracts

Command (the unit of undo)

```
// Single field edit
{ changes: [{ field: 'name', from: '', to: 'Ash' }], at: 1730540000123, mergeKey: 'name' }

// Composite command (Reset)
{ changes: [
    { field: 'name', from: 'Ash', to: '' },
    { field: 'role', from: 'backend', to: 'frontend' },
  ], at: 1730540009000, mergeKey: null }
```

- `changes` is always an array, so a single edit and a multi-field Reset are the same shape: undo applies every `from`, redo applies every `to`. No `switch` per command type.
- `at` is the time of the last keystroke in the burst; used for merging.
- `mergeKey` is the field name for text inputs, `null` for discrete inputs and Reset (never merge).

Initial values (stable keys = input `name`s)

```
const INITIAL_VALUES = { name: '', email: '', bio: '', role: 'frontend', plan: 'free', newsletter:
false };
```

Reducer actions (contract)

Action	Payload	Effect
`EDIT`	`{ field, value, now, mergeable }`	New command, or merge into the last one
`RESET`	`{ initial, now }`	One composite command, or no-op if clean
`UNDO`	none	Pop `past`, apply `from`, push to `future`
`REDO`	none	Pop `future`, apply `to`, push to `past`

`now` comes from the event handler, not `Date.now()` inside the reducer, so the reducer stays pure (StrictMode runs reducers twice).

Hook contract

```
const {
  values, past, future, canUndo, canRedo, isDirty,
  formRef, handleChange, handleKeyDown, undo, redo, reset,
} = useUndoRedoForm(INITIAL_VALUES);
```

No API: everything is local. Submit just shows the values (a real app would `POST` them).

---

6. Trace the data flow

Interaction: user types "Ash", pauses, presses `Ctrl+Z`.

```
keystroke "A" in <input name="name">
  -> handleChange: dispatch EDIT { field:'name', value:'A', mergeable:true, now:t0 }
  -> reducer: last command is not 'name' -> execute new command {from:'', to:'A'}
     past=[cmd1], future=[], canMerge=true
keystroke "s", "h" (each < 600 ms apart)
  -> EDIT -> same mergeKey, within window, canMerge -> replace cmd1 with {from:'', to:'Ash', at:t2}
     still past=[cmd1]  (one undo step)
Ctrl+Z on the form
  -> handleKeyDown: preventDefault (stop native input undo) -> undo()
  -> undo(): read past.at(-1) to know the field -> dispatch UNDO -> focus <input name="name">
  -> reducer: values.name = cmd1.from = '' ; past=[] ; future=[cmd1] ; canMerge=false
  -> render: input shows '', Undo disabled, Redo enabled, panel shows cmd1 struck through
```

---

7. Important technical challenges

Challenge 1: Snapshots vs commands (memento vs command pattern)

Problem: The naive approach stores a full copy of `values` on every change. A form with 30 fields and a 5 KB textarea, typed in for a minute, creates hundreds of full copies.

Solution: Store **commands** that hold only the diff (`field`, `from`, `to`). Undo applies `from`, redo applies `to`. Memory is proportional to what changed, and a command describes the change in words for the history panel.

```
const apply = (values, changes, side) => {
  const next = { ...values };
  changes.forEach((change) => { next[change.field] = change[side]; });
  return next;
};
// undo: apply(values, cmd.changes, 'from')   redo: apply(values, cmd.changes, 'to')
```

Challenge 2: One undo per word, not per keystroke (coalescing)

Problem: Typing "Ashutosh" creates 8 `onChange` events. Pressing Ctrl+Z 8 times to clear one field is unusable.

Solution: If the new edit is to the same text field, within 600 ms of the last one, and the last thing that happened was an edit (not an undo), **stretch** the last command: keep its original `from`, replace `to`. If the burst ends where it started ("a" then Backspace), drop the command. Discrete inputs (checkbox, radio, select) never merge.

```
const shouldMerge =
  mergeable && state.canMerge && last?.mergeKey === field && now - last.at < MERGE_WINDOW_MS;

if (shouldMerge) {
  const original = last.changes[0].from;
  if (Object.is(original, value)) return { ...state, values, past, canMerge: false }; // net zero: drop
  return { ...state, values, past: [...past, { ...last, changes: [{ field, from: original, to: value
}], at: now }] };
}
```

Why `canMerge`: after Undo, the top of `past` is an *older* command for the same field with a recent `at`. Without the flag, the next keystroke would silently rewrite that older command.

Challenge 3: Branching history and multi-field commands

Problem: After undoing twice, the user types. What happens to the redo stack? And how does Reset (which changes many fields) undo in one step?

Solution: Every new command goes through one `execute` function that pushes to `past`, **clears** `future` and caps the length. Reset builds a **composite command** with one change per dirty field, so it reuses the same

apply / undo / redo path.

```
const execute = (state, command) => ({
  values: apply(state.values, command.changes, 'to'),
  past: [...state.past, command].slice(-HISTORY_LIMIT),
  future: [],
  canMerge: true,
});
```

Challenge 4: Shortcuts that fight the browser, and lost focus

Problem: Inputs have their own native undo. Pressing Ctrl+Z inside an input runs the browser's undo, which changes the DOM value behind React's back and goes out of sync with our history. Also, after clicking Undo the user can't see which field changed.

Solution: Handle `onKeyDown` on the `<form>` (events bubble from every field), `preventDefault`, and run our undo. Scoping to the form means other forms / editors on the page are unaffected. After undo / redo, focus the field from the command (`form.elements.namedItem(name)` ; radio groups return a `RadioNodeList`).

```
const handleKeyDown = (e) => {
  if (!(e.ctrlKey || e.metaKey)) return;
  const key = e.key.toLowerCase();
  if (key === 'z' && !e.shiftKey) { e.preventDefault(); undo(); }
  else if ((key === 'z' && e.shiftKey) || key === 'y') { e.preventDefault(); redo(); }
};
```

---

8. Implement in the correct order

#	Step	Working result
1	Scaffold Vite + React + Tailwind, render `App`	Blank page runs
2	Controlled form with `useState` and one generic `handleChange` (`name` / `checked`)	All fields editable
3	Write `historyReducer` with `EDIT` (no merge), `UNDO` , `REDO` ; swap to `useReducer`	Undo / redo works per keystroke
4	Undo / Redo buttons with `disabled` from derived `canUndo` / `canRedo`	Usable UI
5	Clear `future` on new edit; ignore no-op edits	Correct branching
6	Add typing merge (`mergeKey` , `at` , `canMerge`), drop net-zero bursts	One step per word
7	Add `RESET` as a composite command, `isDirty`	Undoable reset
8	Keyboard shortcuts on the form with `preventDefault`	Ctrl+Z / Ctrl+Y
9	Move logic into `useUndoRedoForm` , focus the changed field	Reusable for any form

#	Step	Working result
10	`HistoryPanel`, history cap, a11y attributes, submit preview	Polished demo
11	Node asserts on the reducer (merge, branch, reset, cap)	Confidence

9. Cover all states and edge cases

State / edge case	Handling
Initial	Empty history; Undo, Redo, Reset disabled; "No changes"
Editing	Commands pushed / merged; Undo enabled; "Unsaved changes"
After undo	Redo enabled; focus on the changed field
After new edit post-undo	Redo cleared
Empty history	`UNDO` / `REDO` return the same state (no re-render)
No-op edit (same value)	Returns the same state; no history entry
Burst typed back to original	Command dropped
Fast edits across two fields	Different `mergeKey`, so two commands
Checkbox / radio / select	Never merged; one step each
Reset when clean	Disabled, and reducer no-ops if called anyway
Undo after reset	All fields restored in one step
Edited back to initial manually	`isDirty` false (derived from values, not from history)
More than 100 steps	Oldest dropped (`slice(-HISTORY_LIMIT)`)
Submit	Shows JSON; history kept
Ctrl+Z outside the form	Not handled (scoped listener), browser default applies
React StrictMode double reducer call	Safe: reducer is pure, time comes from the action

10. Final structure

```
_79_undo_redo_form/
├── index.html
├── vite.config.js
├── package.json
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx
    ├── history/
    │   └── historyReducer.js      # pure command / stack logic
```

```
    ├── hooks/
    │   └── useUndoRedoForm.js     # React binding, shortcuts, focus
    └── components/
        ├── ProfileForm.jsx        # the concrete form + toolbar
        └── HistoryPanel.jsx       # applied / undone commands
```

---

11. Full working code

Setup: `npm create vite@latest _79_undo_redo_form -- --template react-swc`, then `npm i tailwindcss @tailwindcss/vite`.

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

src/index.css

```
@import "tailwindcss";

body {
  @apply bg-slate-100 text-slate-900;
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

src/history/historyReducer.js

```
// Command = { changes: [{ field, from, to }], at, mergeKey }
// A command only stores the diff, so undo = apply `from`, redo = apply `to`.
export const HISTORY_LIMIT = 100;
export const MERGE_WINDOW_MS = 600;

export const createHistory = (values) => ({ values, past: [], future: [], canMerge: false });

const apply = (values, changes, side) => {
  const next = { ...values };
  changes.forEach((change) => {
    next[change.field] = change[side];
  });
  return next;
};

// Run a new command: apply it, push it, and drop the redo branch.
const execute = (state, command) => ({
  values: apply(state.values, command.changes, 'to'),
  past: [...state.past, command].slice(-HISTORY_LIMIT),
  future: [],
  canMerge: true,
});

export function historyReducer(state, action) {
  switch (action.type) {
    case 'EDIT': {
      const { field, value, now, mergeable } = action;
      const from = state.values[field];
      if (Object.is(from, value)) return state;

      const last = state.past.at(-1);
      const shouldMerge =
        mergeable && state.canMerge && last?.mergeKey === field && now - last.at < MERGE_WINDOW_MS;

      if (!shouldMerge) {
        return execute(state, { changes: [{ field, from, to: value }], at: now, mergeKey: mergeable ?
field : null });
      }

      // Same field, still typing: stretch the last command instead of adding one per keystroke.
      const original = last.changes[0].from;
      const past = state.past.slice(0, -1);
      const values = { ...state.values, [field]: value };
      // Typed back to where the burst started: the command is a no-op, drop it.
      if (Object.is(original, value)) return { ...state, values, past, canMerge: false };
      return { ...state, values, past: [...past, { ...last, changes: [{ field, from: original, to:
value }], at: now }] };
```

```
    }

    case 'RESET': {
      // One composite command, so a single undo brings every field back.
      const changes = Object.keys(action.initial)
        .filter((field) => !Object.is(state.values[field], action.initial[field]))
        .map((field) => ({ field, from: state.values[field], to: action.initial[field] }));
      if (changes.length === 0) return state;
      return execute(state, { changes, at: action.now, mergeKey: null });
    }

    case 'UNDO': {
      const command = state.past.at(-1);
      if (!command) return state;
      return {
        values: apply(state.values, command.changes, 'from'),
        past: state.past.slice(0, -1),
        future: [...state.future, command],
        canMerge: false,
      };
    }

    case 'REDO': {
      const command = state.future.at(-1);
      if (!command) return state;
      return {
        values: apply(state.values, command.changes, 'to'),
        past: [...state.past, command],
        future: state.future.slice(0, -1),
        canMerge: false,
      };
    }

    default:
      return state;
  }
}
```

src/hooks/useUndoRedoForm.js

```
import { useReducer, useRef } from 'react';
import { createHistory, historyReducer } from '../history/historyReducer';

// Typing in these merges into one command; checkbox / radio / select are one command per change.
const TEXT_TYPES = new Set(['text', 'email', 'password', 'search', 'tel', 'url', 'number',
'textarea']);
```

```
export function useUndoRedoForm(initialValues) {
  const [state, dispatch] = useReducer(historyReducer, initialValues, createHistory);
  const formRef = useRef(null);

  // Move focus to the field that changed, so the user sees what undo/redo did.
  const focusField = (field) => {
    const el = formRef.current?.elements.namedItem(field);
    (el instanceof RadioNodeList ? el[0] : el)?.focus();
  };

  const handleChange = (e) => {
    const { name, type, value, checked } = e.target;
    dispatch({
      type: 'EDIT',
      field: name,
      value: type === 'checkbox' ? checked : value,
      mergeable: TEXT_TYPES.has(type),
      now: Date.now(),
    });
  };

  const undo = () => {
    const command = state.past.at(-1);
    if (!command) return;
    dispatch({ type: 'UNDO' });
    focusField(command.changes[0].field);
  };

  const redo = () => {
    const command = state.future.at(-1);
    if (!command) return;
    dispatch({ type: 'REDO' });
    focusField(command.changes[0].field);
  };

  const reset = () => dispatch({ type: 'RESET', initial: initialValues, now: Date.now() });

  // Scoped to this form: preventDefault stops the browser's own per-input undo from fighting ours.
  const handleKeyDown = (e) => {
    if (!(e.ctrlKey || e.metaKey)) return;
    const key = e.key.toLowerCase();
    if (key === 'z' && !e.shiftKey) {
      e.preventDefault();
      undo();
    } else if ((key === 'z' && e.shiftKey) || key === 'y') {
      e.preventDefault();
      redo();
```

```
    }
  };

  const isDirty = Object.keys(initialValues).some((field) => !Object.is(state.values[field],
initialValues[field]));

  return {
    values: state.values,
    past: state.past,
    future: state.future,
    canUndo: state.past.length > 0,
    canRedo: state.future.length > 0,
    isDirty,
    formRef,
    handleChange,
    handleKeyDown,
    undo,
    redo,
    reset,
  };
}
```

src/components/HistoryPanel.jsx

```
const show = (value) => (typeof value === 'string' ? `"${value.length > 18 ? `${value.slice(0, 18)}…` :
value}"` : String(value));

const describe = (command) =>
  command.changes.length > 1
    ? `Reset ${command.changes.length} fields`
    : `${command.changes[0].field}: ${show(command.changes[0].from)} → ${show(command.changes[0].to)}`;

export default function HistoryPanel({ past, future }) {
  if (past.length === 0 && future.length === 0) {
    return <p className="text-sm text-slate-500">No changes yet. Start typing.</p>;
  }

  return (
    <ol className="space-y-1 text-sm font-mono">
      {past.map((command, i) => (
        <li key={`p${i}`} className={i === past.length - 1 ? 'font-semibold text-indigo-700' : 'text-
slate-700'}>
          {describe(command)}
        </li>
      ))}
      {/* future is a stack: the next redo is its last item, so show it reversed */}
      {[...future].reverse().map((command, i) => (
```

```
        <li key={`f${i}`} className="text-slate-400 line-through">
          {describe(command)}
        </li>
      ))}
    </ol>
  );
}
```

src/components/ProfileForm.jsx

```
import { useState } from 'react';
import { useUndoRedoForm } from '../hooks/useUndoRedoForm';
import HistoryPanel from './HistoryPanel';

const INITIAL_VALUES = { name: '', email: '', bio: '', role: 'frontend', plan: 'free', newsletter:
false };
const ROLES = [
  { value: 'frontend', label: 'Frontend' },
  { value: 'backend', label: 'Backend' },
  { value: 'fullstack', label: 'Full stack' },
];
const PLANS = ['free', 'pro', 'team'];

const inputClass = 'w-full rounded-md border border-slate-300 px-3 py-2 focus:outline-none focus:ring-2
focus:ring-indigo-500';
const buttonClass = 'rounded-md border border-slate-300 px-3 py-1.5 text-sm font-medium hover:bg-slate-
100 disabled:cursor-not-allowed disabled:opacity-40';

export default function ProfileForm() {
  const { values, past, future, canUndo, canRedo, isDirty, formRef, handleChange, handleKeyDown, undo,
redo, reset } =
    useUndoRedoForm(INITIAL_VALUES);
  const [submitted, setSubmitted] = useState(null);

  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmitted(values);
  };

  return (
    <div className="grid gap-6 md:grid-cols-[1fr_280px]">
      <form ref={formRef} onSubmit={handleSubmit} onKeyDown={handleKeyDown} className="space-y-4
rounded-xl bg-white p-6 shadow">
        <div className="flex flex-wrap items-center gap-2" role="toolbar" aria-label="History">
          <button type="button" onClick={undo} disabled={!canUndo} aria-keyshortcuts="Control+Z"
className={buttonClass}>
            Undo
```

```
          </button>
          <button type="button" onClick={redo} disabled={!canRedo} aria-keyshortcuts="Control+Shift+Z
Control+Y" className={buttonClass}>
            Redo
          </button>
          <button type="button" onClick={reset} disabled={!isDirty} className={buttonClass}>
            Reset
          </button>
          <span className="ml-auto text-xs text-slate-500" aria-live="polite">
            {isDirty ? 'Unsaved changes' : 'No changes'}
          </span>
        </div>

        <label className="block space-y-1">
          <span className="text-sm font-medium">Name</span>
          <input name="name" type="text" value={values.name} onChange={handleChange} className=
{inputClass} />
        </label>

        <label className="block space-y-1">
          <span className="text-sm font-medium">Email</span>
          <input name="email" type="email" value={values.email} onChange={handleChange} className=
{inputClass} />
        </label>

        <label className="block space-y-1">
          <span className="text-sm font-medium">Bio</span>
          <textarea name="bio" rows={3} value={values.bio} onChange={handleChange} className=
{inputClass} />
        </label>

        <label className="block space-y-1">
          <span className="text-sm font-medium">Role</span>
          <select name="role" value={values.role} onChange={handleChange} className={inputClass}>
            {ROLES.map((role) => (
              <option key={role.value} value={role.value}>
                {role.label}
              </option>
            ))}
          </select>
        </label>

        <fieldset className="space-y-1">
          <legend className="text-sm font-medium">Plan</legend>
          <div className="flex gap-4">
            {PLANS.map((plan) => (
              <label key={plan} className="flex items-center gap-1 capitalize">
```

```
                <input type="radio" name="plan" value={plan} checked={values.plan === plan} onChange=
{handleChange} />
                {plan}
              </label>
            ))}
          </div>
        </fieldset>

        <label className="flex items-center gap-2">
          <input name="newsletter" type="checkbox" checked={values.newsletter} onChange={handleChange}
/>
          <span className="text-sm">Subscribe to newsletter</span>
        </label>

        <button type="submit" className="rounded-md bg-indigo-600 px-4 py-2 font-medium text-white
hover:bg-indigo-700">
          Save
        </button>

        {submitted && (
          <pre className="overflow-auto rounded-md bg-slate-900 p-3 text-xs text-green-300">
{JSON.stringify(submitted, null, 2)}</pre>
        )}
      </form>

      <aside className="rounded-xl bg-white p-4 shadow">
        <h2 className="mb-2 font-semibold">History</h2>
        <HistoryPanel past={past} future={future} />
      </aside>
    </div>
  );
}
```

src/App.jsx

```
import ProfileForm from './components/ProfileForm';

export default function App() {
  return (
    <main className="mx-auto max-w-4xl p-6">
      <h1 className="mb-1 text-2xl font-bold">Undo / Redo Form</h1>
      <p className="mb-6 text-sm text-slate-600">Ctrl+Z to undo, Ctrl+Shift+Z or Ctrl+Y to redo.</p>
      <ProfileForm />
    </main>
  );
}
```

Reducer self-check (run with `node check.mjs`)

```
import assert from 'node:assert/strict';
import { createHistory, historyReducer as r, HISTORY_LIMIT } from './src/history/historyReducer.js';

const init = { name: '', role: 'fe', news: false };
const edit = (field, value, now, mergeable = true) => ({ type: 'EDIT', field, value, now, mergeable });
let s = createHistory(init);

// typing burst merges into one command
s = r(s, edit('name', 'A', 0)); s = r(s, edit('name', 'As', 100)); s = r(s, edit('name', 'Ash', 200));
assert.equal(s.past.length, 1); assert.deepEqual(s.past[0].changes, [{ field: 'name', from: '', to:
'Ash' }]);
// pause starts a new command
s = r(s, edit('name', 'Ashu', 2000)); assert.equal(s.past.length, 2);
s = r(s, { type: 'UNDO' }); assert.equal(s.values.name, 'Ash');
s = r(s, { type: 'UNDO' }); assert.equal(s.values.name, ''); assert.equal(s.future.length, 2);
s = r(s, { type: 'REDO' }); assert.equal(s.values.name, 'Ash');
// typing right after undo must not merge into the older command
s = r(s, edit('name', 'Ashx', 2100)); assert.equal(s.past.length, 2); assert.equal(s.future.length, 0,
'new edit clears redo');
s = r(s, { type: 'UNDO' }); assert.equal(s.values.name, 'Ash');
// typing back to the start drops the command
s = createHistory(init); s = r(s, edit('name', 'a', 0)); s = r(s, edit('name', '', 50));
assert.equal(s.past.length, 0); assert.equal(s.values.name, '');
// discrete fields never merge
s = r(s, edit('news', true, 0, false)); s = r(s, edit('news', false, 10, false));
assert.equal(s.past.length, 2);
// no-op edit keeps the same state object
assert.equal(r(s, edit('role', 'fe', 20, false)), s);
// reset is one composite command
s = r(s, edit('role', 'be', 30, false)); s = r(s, edit('name', 'Zed', 40));
s = r(s, { type: 'RESET', initial: init, now: 50 }); assert.deepEqual(s.values, init);
s = r(s, { type: 'UNDO' }); assert.deepEqual(s.values, { name: 'Zed', role: 'be', news: false });
assert.equal(r(createHistory(init), { type: 'RESET', initial: init, now: 0 }).past.length, 0);
// undo/redo on empty stacks are no-ops
const empty = createHistory(init); assert.equal(r(empty, { type: 'UNDO' }), empty);
assert.equal(r(empty, { type: 'REDO' }), empty);
// history is capped
s = createHistory(init); for (let i = 0; i < 150; i++) s = r(s, edit('news', i % 2 === 0, i, false));
assert.equal(s.past.length, HISTORY_LIMIT);
console.log('all checks passed');
```

---

12. Wrap-up

Edge cases handled

- Keystroke bursts become one step; bursts that cancel out leave no trace.
- Undo then type never corrupts an older command (`canMerge`).
- New edits clear redo; Reset is one composite, undoable command.
- No-op edits, empty-stack undo / redo, and clean-form reset are all no-ops.
- History is capped; native input undo can't desync React state.

Trade-offs to mention

- **Commands vs snapshots**: commands are smaller and self-describing; snapshots are simpler and safer if the state shape is deeply nested. For a flat form, commands win.
- **Time-based merging** (600 ms) is simple but not word-aware; editors like VS Code also break groups on whitespace or cursor jumps.
- **Overriding native undo** gives one consistent history across fields, but loses the browser's in-field caret restore.
- **Arrays copied per action** (`[...past, cmd]`): O(n) with n ≤ 100, negligible; a linked list or ring buffer only matters for huge histories.
- **Index keys in HistoryPanel**: fine because items are only pushed / popped at the end; with reordering I'd add an `id` per command.

What to add with more time

- Persist history to `sessionStorage` so a refresh keeps it.
- Validation with errors derived from `values` (so undo "restores" errors for free).
- Jump to any history entry (apply undo k times).
- Word-boundary grouping and caret restore (`selectionStart`) on undo.
- Unit tests with Vitest + Testing Library for the hook and keyboard flows.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why is the Command pattern a good fit here?

Every change becomes an object that knows how to apply and reverse itself (`from` / `to`). The history is just two stacks of these objects, and undo / redo / reset all share one code path. New actions (e.g. "fill from template") are just new commands.

Q2. How would you make this work for "any form", e.g. one you don't own?

The hook only needs input `name` s and an initial values object. Wire `onChange={handleChange}` and `onKeyDown` on the form. For uncontrolled third-party forms, listen to `input` events on the form and read `e.target.name / value` the same way.

Q3. What if two forms on the page each need their own undo?

Each calls `useUndoRedoForm` separately, so each has its own stacks, and the shortcut listener is on each `<form>` , not `window` . Focus decides which form handles Ctrl+Z.

State design

Q4. Why `useReducer` instead of several `useState`s?

Undo changes `values`, `past` and `future` together. One reducer makes it one atomic transition, keeps the logic pure and testable in Node, and avoids stale-closure bugs between separate setters.

Q5. Why is `isDirty` not `past.length > 0`?

If the user changes Name to "A" and then back to "", history has entries but the form equals the initial values. Dirty must be derived from values vs initial values.

Q6. Why pass `now` in the action instead of calling `Date.now()` in the reducer?

Reducers must be pure: StrictMode calls them twice and replays must give the same result. Time is an input, so it comes from the event handler. It also makes the merge logic testable with fake times.

Hard parts

Q7. Why not just store a snapshot of the whole form on each change?

It works and is simpler, but costs O(fields) memory per step, and you can't describe what a step did without diffing. Commands store only the diff. I'd pick snapshots if the state were deeply nested and diffing were hard (or use Immer patches).

Q8. How does typing merge exactly, and what breaks the group?

Same field, text-like input, < 600 ms since the last keystroke, and the last action was an edit. A pause, switching fields, an undo / redo, or a discrete input breaks the group.

Q9. What happens if the user types, undoes, and types again in the same field quickly?

`UNDO` sets `canMerge = false`, so the next keystroke creates a new command and clears the redo stack. Without the flag it would merge into the older command and corrupt history.

Q10. Why does Reset need to be a single command?

Users think of Reset as one action. If it were one command per field, they'd press Ctrl+Z six times to get their work back. The composite `changes` array makes it one step.

Q11. Why `preventDefault` on Ctrl+Z? What do we lose?

Native input undo mutates the DOM value directly; React then sees a change event and our history and the browser's history diverge. We lose the native caret positioning, which we could restore by storing `selectionStart` in the command.

Implementation steps

Q12. How do you focus the field after undo when it's a radio group?

`form.elements.namedItem('plan')` returns a `RadioNodeList`, so I focus its first item. Better: focus the item whose value equals the restored value.

Q13. Why read `past.at(-1)` in `undo()` before dispatching?

The dispatch result isn't available synchronously. The top of `past` *is* the command that will be undone, so reading it first tells me which field to focus.

Q14. Is using the array index as a key in HistoryPanel safe?

Here yes: the lists only grow or shrink at the end, so an index always refers to the same command. If we supported deleting or reordering history, each command would get an `id`.

Performance and "what if"

Q15. What if the form has 500 fields?

Each keystroke still creates or updates one small command and copies `values` once (O(fields)). The bigger cost is re-rendering 500 inputs; I'd split fields into memoized components that receive only their own value, or move to uncontrolled inputs with a ref-based store.

Q16. What if the textarea holds 1 MB of text?

Each command stores two full strings (`from` / `to`), so 100 steps can reach 200 MB. I'd store a text diff (e.g. changed range + inserted / removed text) for long strings, or lower the history limit for that field.

Q17. What if undo must survive a refresh or sync across tabs?

Commands are plain JSON, so persist `{ values, past, future }` to `sessionStorage` on change and use it as the reducer's init. For cross-tab, broadcast commands via `BroadcastChannel`.

Q18. What if multiple users edit the same form (collaboration)?

A global stack breaks: undoing must only revert *my* changes. Keep a per-user stack and, on undo, check the field still holds my `to` value; if someone else changed it, skip or transform the command (that is the idea behind OT / CRDT undo).

Q19. How would you support async commands, e.g. "Save" that hits an API?

Saving isn't usually undoable; it's a checkpoint. If it must be, the command needs `execute` / `undo` functions that call the API (e.g. a revert endpoint), with pending and failed states so a failed undo doesn't move the stacks.

Trade-offs

Q20. Why not use a library like `use-undo` or Redux-undo?

They're fine in production, but they snapshot the whole state and don't do keystroke grouping or composite commands out of the box. The interview wants the pattern, and the core is ~80 lines.

Q21. How would you test this?

The reducer is pure, so most behaviour is plain unit tests (merge, branch, reset, cap, no-ops), like `check.mjs`. Then a few Testing Library tests for the hook: type, press Ctrl+Z, assert value and focus.